

You:

Write a Java program using OOP for a coffee shop rewards system. I want the program to have customers, drinks, and orders. Customers should be able to buy drinks and reward points based on what they purchase. I want to use a HashMap to store customers by their customer ID, so the program can find a customer easily. Also I want to use an ArrayList to store the drink menu. The program should allow a customer to check their points and redeem points for a reward.

Response:

Pensado durante un par de segundos

Here is a complete Java OOP program for a **Coffee Shop Rewards System** using:

- Customer class
- Drink class
- Order class
- HashMap to store customers by ID
- ArrayList to store the drink menu
- reward points and redeem system

```
import java.util.*;
```

```
public class CoffeeShopRewardsSystem {  
    public static void main(String[] args) {  
        Scanner console = new Scanner(System.in);  
  
        ArrayList<Drink> menu = new ArrayList<>();  
        menu.add(new Drink("Latte", 4.50));  
        menu.add(new Drink("Americano", 3.25));  
        menu.add(new Drink("Matcha Latte", 5.00));  
        menu.add(new Drink("Mocha", 4.75));  
        menu.add(new Drink("Iced Coffee", 3.50));  
  
        HashMap<Integer, Customer> customers = new HashMap<>();
```

```
customers.put(101, new Customer(101, "Lily"));
customers.put(102, new Customer(102, "Alex"));
customers.put(103, new Customer(103, "Maria"));
```

```
boolean running = true;
```

```
while (running) {
    System.out.println("\n=== Coffee Shop Rewards System ===");
    System.out.println("1. Show drink menu");
    System.out.println("2. Buy a drink");
    System.out.println("3. Check customer points");
    System.out.println("4. Redeem points");
    System.out.println("5. Add new customer");
    System.out.println("6. Exit");
    System.out.print("Choose an option: ");
```

```
    int choice = console.nextInt();
```

```
    if (choice == 1) {
        showMenu(menu);
```

```
    } else if (choice == 2) {
        System.out.print("Enter customer ID: ");
        int id = console.nextInt();
```

```
        if (customers.containsKey(id)) {
            Customer customer = customers.get(id);
```

```
            showMenu(menu);
            System.out.print("Choose a drink number: ");
            int drinkChoice = console.nextInt();
```

```
            if (drinkChoice >= 1 && drinkChoice <= menu.size()) {
                Drink selectedDrink = menu.get(drinkChoice - 1);
                Order order = new Order(customer, selectedDrink);
                order.processOrder();
```

```
            } else {
                System.out.println("Invalid drink choice.");
```

```

    }

    } else {
        System.out.println("Customer not found.");
    }

    } else if (choice == 3) {
        System.out.print("Enter customer ID: ");
        int id = console.nextInt();

        if (customers.containsKey(id)) {
            Customer customer = customers.get(id);
            System.out.println(customer.getName() + " has " + customer.getPoints() + "
points.");
        } else {
            System.out.println("Customer not found.");
        }

    } else if (choice == 4) {
        System.out.print("Enter customer ID: ");
        int id = console.nextInt();

        if (customers.containsKey(id)) {
            Customer customer = customers.get(id);
            customer.redeemReward();
        } else {
            System.out.println("Customer not found.");
        }

    }

    } else if (choice == 5) {
        System.out.print("Enter new customer ID: ");
        int id = console.nextInt();
        console.nextLine();

        if (customers.containsKey(id)) {
            System.out.println("A customer with this ID already exists.");
        } else {
            System.out.print("Enter customer name: ");

```

```

        String name = console.nextLine();

        customers.put(id, new Customer(id, name));
        System.out.println("Customer added successfully.");
    }

    } else if (choice == 6) {
        running = false;
        System.out.println("Thank you for using the Coffee Shop Rewards System!");

    } else {
        System.out.println("Invalid option.");
    }
}

public static void showMenu(ArrayList<Drink> menu) {
    System.out.println("\n=== Drink Menu ===");

    for (int i = 0; i < menu.size(); i++) {
        System.out.println((i + 1) + ". " + menu.get(i));
    }
}

class Customer {
    private int customerId;
    private String name;
    private int points;

    public Customer(int customerId, String name) {
        this.customerId = customerId;
        this.name = name;
        this.points = 0;
    }

    public int getCustomerId() {
        return customerId;
    }
}

```

```

    }

    public String getName() {
        return name;
    }

    public int getPoints() {
        return points;
    }

    public void addPoints(int pointsEarned) {
        points += pointsEarned;
    }

    public void redeemReward() {
        int rewardCost = 50;

        if (points >= rewardCost) {
            points -= rewardCost;
            System.out.println(name + " redeemed 50 points for a free drink reward!");
            System.out.println("Remaining points: " + points);
        } else {
            System.out.println(name + " does not have enough points to redeem a reward.");
            System.out.println("Current points: " + points);
            System.out.println("Points needed: " + rewardCost);
        }
    }

    public String toString() {
        return "Customer ID: " + customerId + ", Name: " + name + ", Points: " + points;
    }
}

class Drink {
    private String name;
    private double price;

    public Drink(String name, double price) {

```

```

        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public double getPrice() {
        return price;
    }

    public int calculatePoints() {
        return (int)(price * 10);
    }

    public String toString() {
        return name + " - $" + price;
    }
}

class Order {
    private Customer customer;
    private Drink drink;

    public Order(Customer customer, Drink drink) {
        this.customer = customer;
        this.drink = drink;
    }

    public void processOrder() {
        int pointsEarned = drink.calculatePoints();
        customer.addPoints(pointsEarned);

        System.out.println("\nOrder completed!");
        System.out.println("Customer: " + customer.getName());
        System.out.println("Drink: " + drink.getName());
        System.out.println("Price: $" + drink.getPrice());
    }
}

```

```
        System.out.println("Points earned: " + pointsEarned);
        System.out.println("Total points: " + customer.getPoints());
    }
}
```

This is a good initial version because it clearly shows OOP design and uses both required data structures: `HashMap<Integer, Customer>` and `ArrayList<Drink>`.

You:

Can you separate those in different files

Response:

Yes — you can separate it into **4 different files**:

1. `CoffeeShopRewardsSystem.java`
2. `Customer.java`
3. `Drink.java`
4. `Order.java`

Make sure all files are in the **same folder**.

CoffeeShopRewardsSystem.java

```
import java.util.*;

public class CoffeeShopRewardsSystem {
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);

        ArrayList<Drink> menu = new ArrayList<>();
        menu.add(new Drink("Latte", 4.50));
        menu.add(new Drink("Americano", 3.25));
        menu.add(new Drink("Matcha Latte", 5.00));
        menu.add(new Drink("Mocha", 4.75));
        menu.add(new Drink("Iced Coffee", 3.50));
```

```
HashMap<Integer, Customer> customers = new HashMap<>();
```

```
customers.put(101, new Customer(101, "Lily"));  
customers.put(102, new Customer(102, "Alex"));  
customers.put(103, new Customer(103, "Maria"));
```

```
boolean running = true;
```

```
while (running) {  
    System.out.println("\n=== Coffee Shop Rewards System ===");  
    System.out.println("1. Show drink menu");  
    System.out.println("2. Buy a drink");  
    System.out.println("3. Check customer points");  
    System.out.println("4. Redeem points");  
    System.out.println("5. Add new customer");  
    System.out.println("6. Exit");  
    System.out.print("Choose an option: ");
```

```
    int choice = console.nextInt();
```

```
    if (choice == 1) {  
        showMenu(menu);
```

```
    } else if (choice == 2) {  
        System.out.print("Enter customer ID: ");  
        int id = console.nextInt();
```

```
        if (customers.containsKey(id)) {  
            Customer customer = customers.get(id);
```

```
            showMenu(menu);  
            System.out.print("Choose a drink number: ");  
            int drinkChoice = console.nextInt();
```

```
            if (drinkChoice >= 1 && drinkChoice <= menu.size()) {  
                Drink selectedDrink = menu.get(drinkChoice - 1);  
                Order order = new Order(customer, selectedDrink);  
                order.processOrder();
```



```

        } else {
            System.out.println("Invalid drink choice.");
        }

    } else {
        System.out.println("Customer not found.");
    }

} else if (choice == 3) {
    System.out.print("Enter customer ID: ");
    int id = console.nextInt();

    if (customers.containsKey(id)) {
        Customer customer = customers.get(id);
        System.out.println(customer.getName() + " has " + customer.getPoints() + "
points.");
    } else {
        System.out.println("Customer not found.");
    }

} else if (choice == 4) {
    System.out.print("Enter customer ID: ");
    int id = console.nextInt();

    if (customers.containsKey(id)) {
        Customer customer = customers.get(id);
        customer.redeemReward();
    } else {
        System.out.println("Customer not found.");
    }

} else if (choice == 5) {
    System.out.print("Enter new customer ID: ");
    int id = console.nextInt();
    console.nextLine();

    if (customers.containsKey(id)) {
        System.out.println("A customer with this ID already exists.");
    }
}

```

```

    } else {
        System.out.print("Enter customer name: ");
        String name = console.nextLine();

        customers.put(id, new Customer(id, name));
        System.out.println("Customer added successfully.");
    }

} else if (choice == 6) {
    running = false;
    System.out.println("Thank you for using the Coffee Shop Rewards System!");

} else {
    System.out.println("Invalid option.");
}
}
}

public static void showMenu(ArrayList<Drink> menu) {
    System.out.println("\n=== Drink Menu ===");

    for (int i = 0; i < menu.size(); i++) {
        System.out.println((i + 1) + ". " + menu.get(i));
    }
}
}

```

Customer.java

```

public class Customer {
    private int customerId;
    private String name;
    private int points;

    public Customer(int customerId, String name) {
        this.customerId = customerId;
        this.name = name;
    }
}

```

```
        this.points = 0;
    }

    public int getCustomerId() {
        return customerId;
    }

    public String getName() {
        return name;
    }

    public int getPoints() {
        return points;
    }

    public void addPoints(int pointsEarned) {
        points += pointsEarned;
    }

    public void redeemReward() {
        int rewardCost = 50;

        if (points >= rewardCost) {
            points -= rewardCost;
            System.out.println(name + " redeemed 50 points for a free drink reward!");
            System.out.println("Remaining points: " + points);
        } else {
            System.out.println(name + " does not have enough points to redeem a reward.");
            System.out.println("Current points: " + points);
            System.out.println("Points needed: " + rewardCost);
        }
    }

    public String toString() {
        return "Customer ID: " + customerId + ", Name: " + name + ", Points: " + points;
    }
}
```

Drink.java

```
public class Drink {  
    private String name;  
    private double price;  
  
    public Drink(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public double getPrice() {  
        return price;  
    }  
  
    public int calculatePoints() {  
        return (int)(price * 10);  
    }  
  
    public String toString() {  
        return name + " - $" + price;  
    }  
}
```

Order.java

```
public class Order {  
    private Customer customer;  
    private Drink drink;  
  
    public Order(Customer customer, Drink drink) {  
        this.customer = customer;  
    }  
}
```

```

        this.drink = drink;
    }

    public void processOrder() {
        int pointsEarned = drink.calculatePoints();
        customer.addPoints(pointsEarned);

        System.out.println("\nOrder completed!");
        System.out.println("Customer: " + customer.getName());
        System.out.println("Drink: " + drink.getName());
        System.out.println("Price: $" + drink.getPrice());
        System.out.println("Points earned: " + pointsEarned);
        System.out.println("Total points: " + customer.getPoints());
    }
}

```

To run it, compile the main file:

```

javac CoffeeShopRewardsSystem.java
java CoffeeShopRewardsSystem

```

You:

```

=== Drink Menu === 1. Latte - $4.5 2. Americano - $3.25 3. Matcha Latte - $5.0 4. Mocha -
$4.75 5. Iced Coffee - $3.5 === Coffee Shop Rewards System === 1. Show drink menu 2.
Buy a drink 3. Check customer points 4. Redeem points 5. Add new customer 6. Exit
Choose an option: Can you make the Coffee Shop Rewards System stop popping up
constantly so it only appears when I actually want to go back or in the beginning

```

Response:

Yes. Right now the menu appears again after every action because the whole menu is inside the while (running) loop.

To make it appear only at the beginning and when the user wants to go back, you can add a **“Press Enter to return to main menu”** pause after each action. That way, the menu does not immediately pop up.

Replace your CoffeeShopRewardsSystem.java with this version:


```

        showMenu(menu);
        System.out.print("Choose a drink number: ");
        int drinkChoice = console.nextInt();

        if (drinkChoice >= 1 && drinkChoice <= menu.size()) {
            Drink selectedDrink = menu.get(drinkChoice - 1);
            Order order = new Order(customer, selectedDrink);
            order.processOrder();
        } else {
            System.out.println("Invalid drink choice.");
        }

    } else {
        System.out.println("Customer not found.");
    }

    console.nextLine();
    pause(console);

} else if (choice == 3) {
    System.out.print("Enter customer ID: ");
    int id = console.nextInt();

    if (customers.containsKey(id)) {
        Customer customer = customers.get(id);
        System.out.println(customer.getName() + " has " + customer.getPoints() + "
points.");
    } else {
        System.out.println("Customer not found.");
    }

    console.nextLine();
    pause(console);

} else if (choice == 4) {
    System.out.print("Enter customer ID: ");
    int id = console.nextInt();

```

```

        if (customers.containsKey(id)) {
            Customer customer = customers.get(id);
            customer.redeemReward();
        } else {
            System.out.println("Customer not found.");
        }

        console.nextLine();
        pause(console);

    } else if (choice == 5) {
        System.out.print("Enter new customer ID: ");
        int id = console.nextInt();
        console.nextLine();

        if (customers.containsKey(id)) {
            System.out.println("A customer with this ID already exists.");
        } else {
            System.out.print("Enter customer name: ");
            String name = console.nextLine();

            customers.put(id, new Customer(id, name));
            System.out.println("Customer added successfully.");
        }

        pause(console);

    } else if (choice == 6) {
        running = false;
        System.out.println("Thank you for using the Coffee Shop Rewards System!");

    } else {
        System.out.println("Invalid option.");
        pause(console);
    }
}
}

```



```

public static void showMainMenu() {
    System.out.println("\n=== Coffee Shop Rewards System ===");
    System.out.println("1. Show drink menu");
    System.out.println("2. Buy a drink");
    System.out.println("3. Check customer points");
    System.out.println("4. Redeem points");
    System.out.println("5. Add new customer");
    System.out.println("6. Exit");
    System.out.print("Choose an option: ");
}

public static void showMenu(ArrayList<Drink> menu) {
    System.out.println("\n=== Drink Menu ===");

    for (int i = 0; i < menu.size(); i++) {
        System.out.println((i + 1) + ". " + menu.get(i));
    }
}

public static void pause(Scanner console) {
    System.out.println();
    System.out.print("Press Enter to go back to the main menu...");
    console.nextLine();
}
}

```

Now the program will work like this:

=== Coffee Shop Rewards System ===

```

1. Show drink menu
2. Buy a drink
3. Check customer points
4. Redeem points
5. Add new customer
6. Exit
Choose an option: 1

```

=== Drink Menu ===

```

1. Latte - $4.5

```

2. Americano - \$3.25
3. Matcha Latte - \$5.0
4. Mocha - \$4.75
5. Iced Coffee - \$3.5

Press Enter to go back to the main menu...

So the menu only comes back after you press Enter.

You:

can you add another drink to your order if you wish; you can also confirm your order, remove items or adjust the quantity of the drinks you selected, and specify the sugar level.

Response:

I'll extend the order flow so one order can contain multiple drink selections, quantities, and sugar levels before checkout. The main changes will be in Order.java and the "Buy a drink" section of CoffeeShopRewardsSystem.java.

Pensado durante un par de segundos

Yes. The cleanest way is to add one more class: OrderItem.java.

Now the files will be:

CoffeeShopRewardsSystem.java

Customer.java

Drink.java

Order.java

OrderItem.java

Customer.java and Drink.java can stay the same. Replace only CoffeeShopRewardsSystem.java and Order.java, and add the new OrderItem.java.

Updated CoffeeShopRewardsSystem.java

```
import java.util.*;
```

```
public class CoffeeShopRewardsSystem {  
    public static void main(String[] args) {  
        Scanner console = new Scanner(System.in);  
  
        ArrayList<Drink> menu = new ArrayList<>();  
        menu.add(new Drink("Latte", 4.50));  
        menu.add(new Drink("Americano", 3.25));  
        menu.add(new Drink("Matcha Latte", 5.00));  
        menu.add(new Drink("Mocha", 4.75));  
        menu.add(new Drink("Iced Coffee", 3.50));  
  
        HashMap<Integer, Customer> customers = new HashMap<>();  
  
        customers.put(101, new Customer(101, "Lily"));  
        customers.put(102, new Customer(102, "Alex"));  
        customers.put(103, new Customer(103, "Maria"));  
  
        boolean running = true;
```

```
while (running) {  
    showMainMenu();  
  
    int choice = console.nextInt();  
    console.nextLine();  
  
    if (choice == 1) {  
        showMenu(menu);  
        pause(console);  
  
    } else if (choice == 2) {  
        buyDrinks(console, customers, menu);  
        pause(console);  
  
    } else if (choice == 3) {  
        System.out.print("Enter customer ID: ");  
        int id = console.nextInt();  
        console.nextLine();  
  
        if (customers.containsKey(id)) {  
            Customer customer = customers.get(id);  
            System.out.println(customer.getName() + " has " + customer.getPoints() + "  
points.");  
        } else {  
            System.out.println("Customer not found.");  
        }  
    }  
}
```

```
    pause(console);
```

```
} else if (choice == 4) {
```

```
    System.out.print("Enter customer ID: ");
```

```
    int id = console.nextInt();
```

```
    console.nextLine();
```

```
    if (customers.containsKey(id)) {
```

```
        Customer customer = customers.get(id);
```

```
        customer.redeemReward();
```

```
    } else {
```

```
        System.out.println("Customer not found.");
```

```
    }
```

```
    pause(console);
```

```
} else if (choice == 5) {
```

```
    System.out.print("Enter new customer ID: ");
```

```
    int id = console.nextInt();
```

```
    console.nextLine();
```

```
    if (customers.containsKey(id)) {
```

```
        System.out.println("A customer with this ID already exists.");
```

```
    } else {
```

```
        System.out.print("Enter customer name: ");
```

```

        String name = console.nextLine();

        customers.put(id, new Customer(id, name));

        System.out.println("Customer added successfully.");
    }

    pause(console);

} else if (choice == 6) {
    running = false;

    System.out.println("Thank you for using the Coffee Shop Rewards System!");

} else {
    System.out.println("Invalid option.");
    pause(console);
}
}

public static void buyDrinks(Scanner console, HashMap<Integer, Customer> customers,
ArrayList<Drink> menu) {
    System.out.print("Enter customer ID: ");

    int id = console.nextInt();
    console.nextLine();

    if (!customers.containsKey(id)) {

```

```
System.out.println("Customer not found.");  
return;  
}
```

```
Customer customer = customers.get(id);  
Order order = new Order(customer);
```

```
boolean ordering = true;
```

```
while (ordering) {  
    System.out.println("\n=== Current Order Menu ===");  
    System.out.println("1. Add drink");  
    System.out.println("2. Remove drink");  
    System.out.println("3. Adjust quantity");  
    System.out.println("4. View order");  
    System.out.println("5. Confirm order");  
    System.out.println("6. Cancel order");  
    System.out.print("Choose an option: ");
```

```
int orderChoice = console.nextInt();  
console.nextLine();
```

```
if (orderChoice == 1) {  
    showMenu(menu);
```

```
System.out.print("Choose a drink number: ");
```

```

int drinkChoice = console.nextInt();

console.nextLine();

if (drinkChoice >= 1 && drinkChoice <= menu.size()) {
    Drink selectedDrink = menu.get(drinkChoice - 1);

    System.out.print("Enter quantity: ");

    int quantity = console.nextInt();
    console.nextLine();

    if (quantity <= 0) {
        System.out.println("Quantity must be greater than 0.");
    } else {
        System.out.print("Enter sugar level, for example 0%, 25%, 50%, 75%, or 100%:");

        String sugarLevel = console.nextLine();

        order.addItem(selectedDrink, quantity, sugarLevel);
        System.out.println("Drink added to order.");
    }

} else {
    System.out.println("Invalid drink choice.");
}

} else if (orderChoice == 2) {

```



```
if (order.isEmpty()) {  
    System.out.println("Your order is currently empty.");  
} else {  
    order.printOrder();  
  
    System.out.print("Enter item number to remove: ");  
    int itemNumber = console.nextInt();  
    console.nextLine();  
  
    order.removeItem(itemNumber);  
}  
  
} else if (orderChoice == 3) {  
    if (order.isEmpty()) {  
        System.out.println("Your order is currently empty.");  
    } else {  
        order.printOrder();  
  
        System.out.print("Enter item number to adjust: ");  
        int itemNumber = console.nextInt();  
        console.nextLine();  
  
        System.out.print("Enter new quantity: ");  
        int newQuantity = console.nextInt();  
        console.nextLine();
```

```
        order.adjustQuantity(itemNumber, newQuantity);
    }

    } else if (orderChoice == 4) {
        order.printOrder();

    } else if (orderChoice == 5) {
        if (order.isEmpty()) {
            System.out.println("You cannot confirm an empty order.");
        } else {
            order.processOrder();
            ordering = false;
        }

    } else if (orderChoice == 6) {
        System.out.println("Order canceled.");
        ordering = false;

    } else {
        System.out.println("Invalid option.");
    }
}

public static void showMainMenu() {
    System.out.println("\n=== Coffee Shop Rewards System ===");
}
```

```
System.out.println("1. Show drink menu");  
System.out.println("2. Buy a drink");  
System.out.println("3. Check customer points");  
System.out.println("4. Redeem points");  
System.out.println("5. Add new customer");  
System.out.println("6. Exit");  
System.out.print("Choose an option: ");  
}
```

```
public static void showMenu(ArrayList<Drink> menu) {  
    System.out.println("\n=== Drink Menu ===");  
  
    for (int i = 0; i < menu.size(); i++) {  
        System.out.println((i + 1) + ". " + menu.get(i));  
    }  
}
```

```
public static void pause(Scanner console) {  
    System.out.println();  
    System.out.print("Press Enter to go back to the main menu...");  
    console.nextLine();  
}  
}
```

Updated Order.java

```
import java.util.*;
```

```
public class Order {  
  
    private Customer customer;  
  
    private ArrayList<OrderItem> items;  
  
    public Order(Customer customer) {  
        this.customer = customer;  
        this.items = new ArrayList<>();  
    }  
  
    public void addItem(Drink drink, int quantity, String sugarLevel) {  
        items.add(new OrderItem(drink, quantity, sugarLevel));  
    }  
  
    public void removeItem(int itemNumber) {  
        int index = itemNumber - 1;  
  
        if (index >= 0 && index < items.size()) {  
            OrderItem removedItem = items.remove(index);  
            System.out.println(removedItem.getDrink().getName() + " was removed from the  
order.");  
        } else {  
            System.out.println("Invalid item number.");  
        }  
    }  
  
    public void adjustQuantity(int itemNumber, int newQuantity) {
```

```
int index = itemNumber - 1;

if (index >= 0 && index < items.size()) {
    if (newQuantity > 0) {
        items.get(index).setQuantity(newQuantity);
        System.out.println("Quantity updated.");
    } else {
        System.out.println("Quantity must be greater than 0.");
    }
} else {
    System.out.println("Invalid item number.");
}
}
```

```
public boolean isEmpty() {
    return items.isEmpty();
}
```

```
public double calculateTotal() {
    double total = 0;

    for (OrderItem item : items) {
        total += item.getSubtotal();
    }
}
```

```
return total;
```

```
}
```

```
public int calculateTotalPoints() {
```

```
    int totalPoints = 0;
```

```
    for (OrderItem item : items) {
```

```
        totalPoints += item.calculatePoints();
```

```
    }
```

```
    return totalPoints;
```

```
}
```

```
public void printOrder() {
```

```
    if (items.isEmpty()) {
```

```
        System.out.println("Your order is currently empty.");
```

```
    } else {
```

```
        System.out.println("\n=== Current Order ===");
```

```
        for (int i = 0; i < items.size(); i++) {
```

```
            System.out.println((i + 1) + ". " + items.get(i));
```

```
        }
```

```
        System.out.printf("Total: $%.2f\n", calculateTotal());
```

```
        System.out.println("Estimated points: " + calculateTotalPoints());
```

```
    }
```

```
}
```

```
public void processOrder() {  
    int pointsEarned = calculateTotalPoints();  
    customer.addPoints(pointsEarned);  
  
    System.out.println("\nOrder confirmed!");  
    System.out.println("Customer: " + customer.getName());  
  
    printOrder();  
  
    System.out.println("Points earned: " + pointsEarned);  
    System.out.println("Total customer points: " + customer.getPoints());  
}  
}
```

New file: OrderItem.java

```
public class OrderItem {  
    private Drink drink;  
    private int quantity;  
    private String sugarLevel;  
  
    public OrderItem(Drink drink, int quantity, String sugarLevel) {  
        this.drink = drink;  
        this.quantity = quantity;  
        this.sugarLevel = sugarLevel;  
    }  
}
```

```
public Drink getDrink() {  
    return drink;  
}
```

```
public int getQuantity() {  
    return quantity;  
}
```

```
public String getSugarLevel() {  
    return sugarLevel;  
}
```

```
public void setQuantity(int quantity) {  
    this.quantity = quantity;  
}
```

```
public double getSubtotal() {  
    return drink.getPrice() * quantity;  
}
```

```
public int calculatePoints() {  
    return drink.calculatePoints() * quantity;  
}
```

```
public String toString() {  
    return drink.getName()
```



```
        + " | Quantity: " + quantity  
        + " | Sugar: " + sugarLevel  
        + " | Subtotal: $" + String.format("%.2f", getSubtotal());  
    }  
}
```

Keep Customer.java the same

```
public class Customer {  
    private int customerId;  
    private String name;  
    private int points;  
  
    public Customer(int customerId, String name) {  
        this.customerId = customerId;  
        this.name = name;  
        this.points = 0;  
    }  
  
    public int getCustomerId() {  
        return customerId;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getPoints() {
```

```

        return points;
    }

    public void addPoints(int pointsEarned) {
        points += pointsEarned;
    }

    public void redeemReward() {
        int rewardCost = 50;

        if (points >= rewardCost) {
            points -= rewardCost;

            System.out.println(name + " redeemed 50 points for a free drink reward!");
            System.out.println("Remaining points: " + points);
        } else {
            System.out.println(name + " does not have enough points to redeem a reward.");
            System.out.println("Current points: " + points);
            System.out.println("Points needed: " + rewardCost);
        }
    }

    public String toString() {
        return "Customer ID: " + customerId + ", Name: " + name + ", Points: " + points;
    }
}

```

Keep Drink.java the same

```
public class Drink {  
    private String name;  
    private double price;  
  
    public Drink(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public double getPrice() {  
        return price;  
    }  
  
    public int calculatePoints() {  
        return (int)(price * 10);  
    }  
  
    public String toString() {  
        return name + " - $" + String.format("%.2f", price);  
    }  
}
```

Now when the customer chooses Buy a drink, they will get this mini-menu:

=== Current Order Menu ===

1. Add drink
2. Remove drink
3. Adjust quantity
4. View order
5. Confirm order
6. Cancel order

So they can add multiple drinks, remove items, change quantities, choose sugar level, and confirm the order only when they are ready.

You:

Could you add the option for when I redeem my points to claim a free drink?

Response:

Yes. Right now your `redeemReward()` only subtracts 50 points and says the customer redeemed a reward, but it does **not let the customer actually choose a free drink**.

To fix that, we should move the redeem process into `CoffeeShopRewardsSystem.java`, because that file has access to the menu.

You only need to update **2 files**:

1. `Customer.java`
2. `CoffeeShopRewardsSystem.java`

Update `Customer.java`

Replace your current `Customer.java` with this version:

```
public class Customer {  
    private int customerId;  
    private String name;  
    private int points;  
  
    public Customer(int customerId, String name) {  
        this.customerId = customerId;  
        this.name = name;  
        this.points = 0;  
    }  
  
    public int getCustomerId() {  
        return customerId;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getPoints() {  
        return points;  
    }  
  
    public void addPoints(int pointsEarned) {  
        points += pointsEarned;  
    }  
  
    public boolean hasEnoughPoints(int rewardCost) {  
        return points >= rewardCost;  
    }  
  
    public void subtractPoints(int pointsUsed) {  
        points -= pointsUsed;  
    }  
  
    public String toString() {  
        return "Customer ID: " + customerId + ", Name: " + name + ", Points: " + points;  
    }  
}
```

```
}  
}
```

Main change: I removed `redeemReward()` and replaced it with:

```
public boolean hasEnoughPoints(int rewardCost)
```

and

```
public void subtractPoints(int pointsUsed)
```

That makes it easier to redeem points for a specific drink from the menu.

Update option 4 in `CoffeeShopRewardsSystem.java`

In your main method, find this part:

```
} else if (choice == 4) {  
    System.out.print("Enter customer ID: ");  
    int id = console.nextInt();  
    console.nextLine();  
  
    if (customers.containsKey(id)) {  
        Customer customer = customers.get(id);  
        customer.redeemReward();  
    } else {  
        System.out.println("Customer not found.");  
    }  
  
    pause(console);  
}
```

Replace it with this:

```
} else if (choice == 4) {  
    redeemFreeDrink(console, customers, menu);  
    pause(console);  
}
```

Then add this method inside `CoffeeShopRewardsSystem.java`

Put this method below your `buyDrinks` method:

```
public static void redeemFreeDrink(Scanner console, HashMap<Integer, Customer>
customers, ArrayList<Drink> menu) {
    int rewardCost = 50;

    System.out.print("Enter customer ID: ");
    int id = console.nextInt();
    console.nextLine();

    if (!customers.containsKey(id)) {
        System.out.println("Customer not found.");
        return;
    }

    Customer customer = customers.get(id);

    System.out.println(customer.getName() + " has " + customer.getPoints() + " points.");
    System.out.println("A free drink reward costs " + rewardCost + " points.");

    if (!customer.hasEnoughPoints(rewardCost)) {
        System.out.println("You do not have enough points to claim a free drink.");
        System.out.println("You need " + (rewardCost - customer.getPoints()) + " more points.");
        return;
    }

    showMenu(menu);

    System.out.print("Choose the free drink you want to claim: ");
    int drinkChoice = console.nextInt();
    console.nextLine();

    if (drinkChoice >= 1 && drinkChoice <= menu.size()) {
        Drink freeDrink = menu.get(drinkChoice - 1);

        System.out.print("Enter sugar level, for example 0%, 25%, 50%, 75%, or 100%: ");
        String sugarLevel = console.nextLine();

        customer.subtractPoints(rewardCost);
```

```
        System.out.println("\nReward claimed successfully!");
        System.out.println("Customer: " + customer.getName());
        System.out.println("Free drink: " + freeDrink.getName());
        System.out.println("Sugar level: " + sugarLevel);
        System.out.println("Points used: " + rewardCost);
        System.out.println("Remaining points: " + customer.getPoints());

    } else {
        System.out.println("Invalid drink choice.");
    }
}
```

Your main menu can stay the same

```
System.out.println("4. Redeem points");
```

But you can make it clearer like this:

```
System.out.println("4. Redeem points for a free drink");
```

Now when the user chooses option 4, the program will work like this:

Enter customer ID: 101

Lily has 80 points.

A free drink reward costs 50 points.

=== Drink Menu ===

1. Latte - \$4.50
2. Americano - \$3.25
3. Matcha Latte - \$5.00
4. Mocha - \$4.75
5. Iced Coffee - \$3.50

Choose the free drink you want to claim: 3

Enter sugar level, for example 0%, 25%, 50%, 75%, or 100%: 50%

Reward claimed successfully!

Customer: Lily

Free drink: Matcha Latte

Sugar level: 50%

Points used: 50

Remaining points: 30

So now redeeming points actually lets the customer **claim a specific free drink**.